

Task 1 -

A setup:

I created a notebook for setup in which 5000 labeled training images were used, and there were 8000 test images and 10 classes. The training set was split: 80% Train, 20% Validation. This produced a set of 4000 training examples and 1000 validation examples. I also displayed the training and validation count of each class, which was 400:100, respectively. This was equally maintained by stratification seed as it made sure that each class received equal training + validation examples. We need 500 class images so each model can receive equal images.

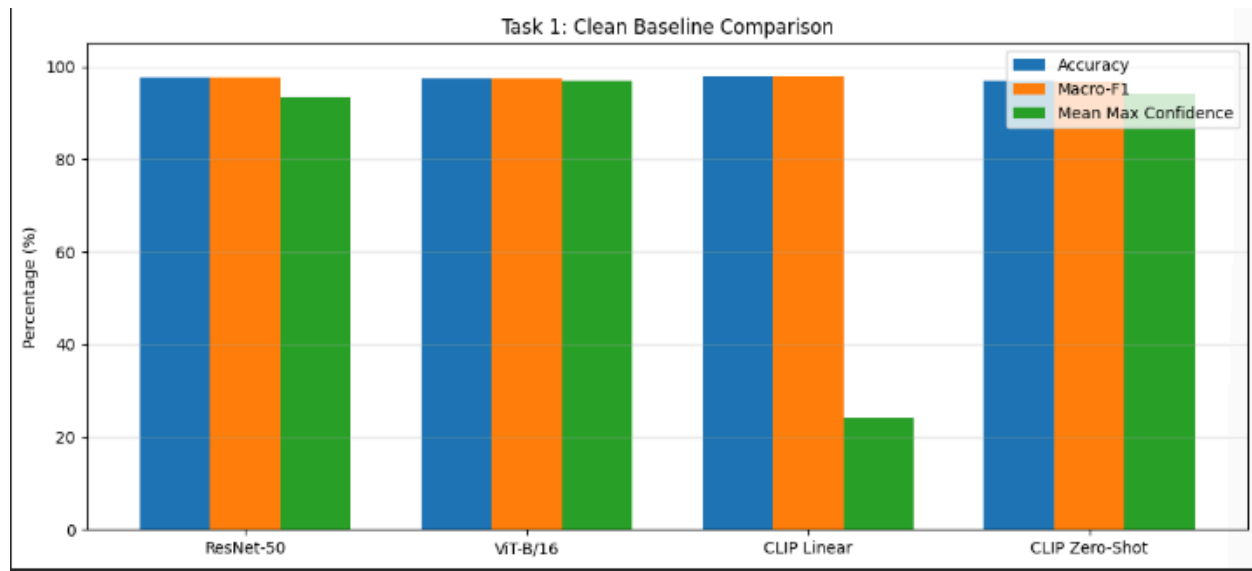
B model:

Another separate notebook was created for models only, where models were installed and optimized according to what's needed in the tasks, e.g., freezing backbone parameters of models. Basically, we are trying to train our models on STL-10. Suppose if accuracy is high, we can't configure whether it is because of the dataset or fine-tuning. To remove this ambiguity, we use Linear Probing, which freezes the entire pretrained model and only trains the tiny classifier layer on top. Also, to produce a correct representation of the dataset that can be accepted by the model. I defined a custom Pytorch dataset that converts STL-10 images from numpy arrays into RGB PIL images, resizes them to 224 x 224, converts them into Pytorch tensors, and then at last creates the split train, validation, and test datasets to be used by pretrained backbones. Optimizer used was AdamW, cross-entropy loss was used, and training was done for almost 50 epochs, but if the validation accuracy wasn't improved after 5 consecutive epochs, training stops. Resnet gave 97.3% best accuracy, ViT-B/16 gave 98.1%, and CLIP linear gave 98.6% validation accuracy. The only anomaly was CLIP's training loss being higher than others, and this was because CLIP selects the class whose text embedding has the greatest cosine similarity to the image.

If a frozen representation gives poor accuracy with a linear probe, this means useful information isn't accessible or is lost, but since each model gave good accuracy, it means that the classes are very organized in feature space.

C-Clean Baseline:

This was used just to load everything already trained and then evaluate everything on exactly the same 500 clean images, and then use the numbers established as a comparison against other experiments. This is basically a controlled experiment. I used 4 models to run the same images, i.e., ResNet, ViT, Clip Linear Probing, and CLIP zero-shot. CLIP zero-shot was used to exploit the special property of CLIP that it places images and text representations together. So, basically, we are zero-shot classifying images using text after testing all 500 images on different models. We use accuracy, Macro-F1, and Max-Mean Confidence to evaluate the model's performance.



Interpretations from this graph:

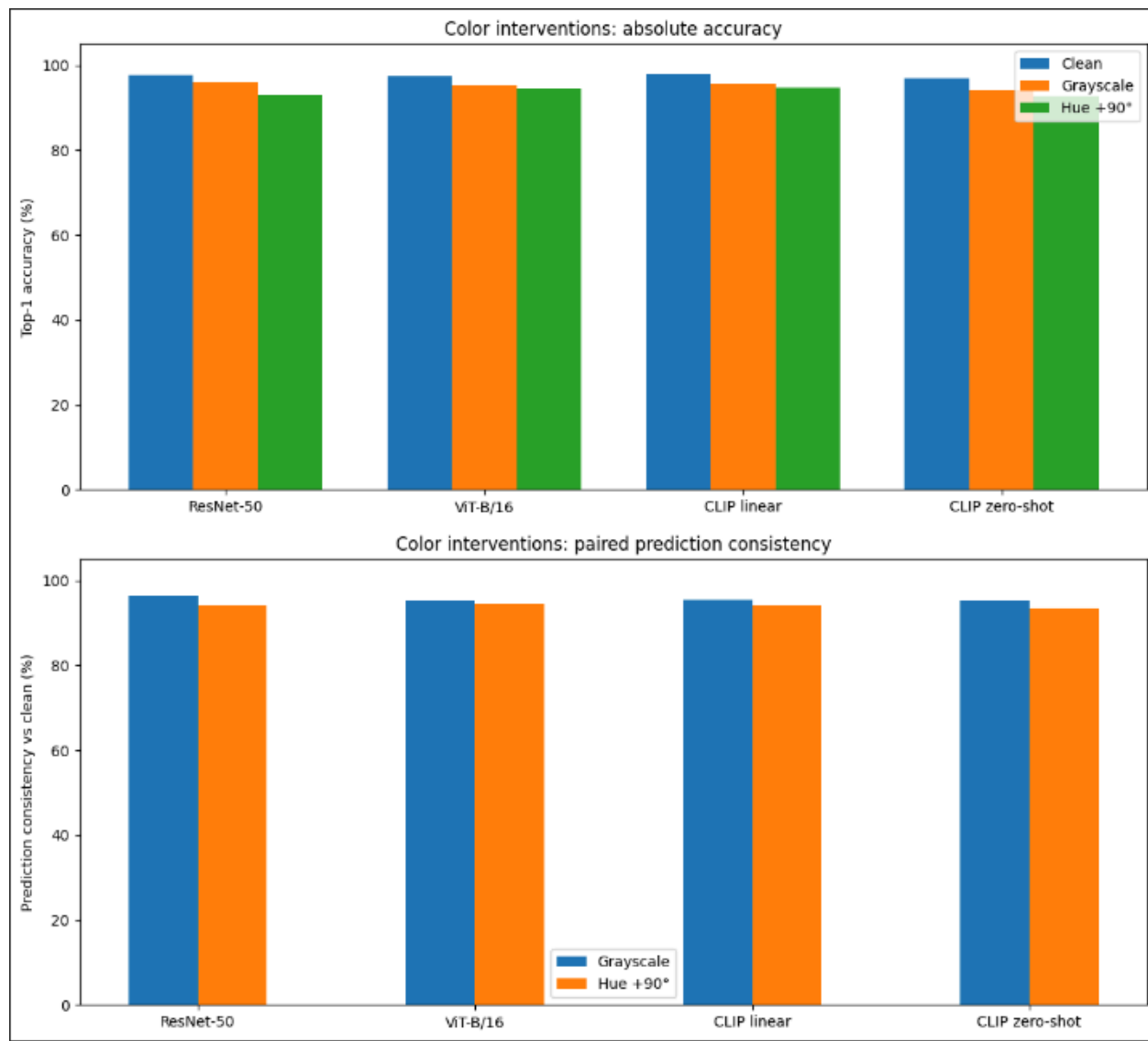
Accuracy and F1 are almost similar. This makes sense because the dataset was perfectly balanced and also suggests errors weren't concentrated in 1 class.

The only anomaly was CLIP's 98% accuracy and 24.15% confidence in linear probing. This is completely justifiable as accuracy depends on which logit is the largest, whereas confidence depends on how much larger that logit is than the others. Since task 2, I stopped training when validation accuracy was the same, but at CLIP linear, even though validation accuracy was the same, loss was higher, which made this confidence approximately low through calculations.

D- Color Bias:

This experiment basically asks a question about how much the model depends on color. We took exactly 500 images and created 2 interventions: Changed it into grayscale and also hue-shifted by 90 degrees.

The conceptual reason for this intervention was that GrayScale removes chromatic influence & information. Everything is present within the image, but the information "this region is red" is removed. Hue rotation basically keeps saturation and brightness the same, but actual color identity changes. A red object becomes greenish. In other words, it provides incorrect color information. Hue rotation is implemented before normalization because it makes less sense to apply hue rotation to normalized tensors whose channels might not represent RGB intensities.

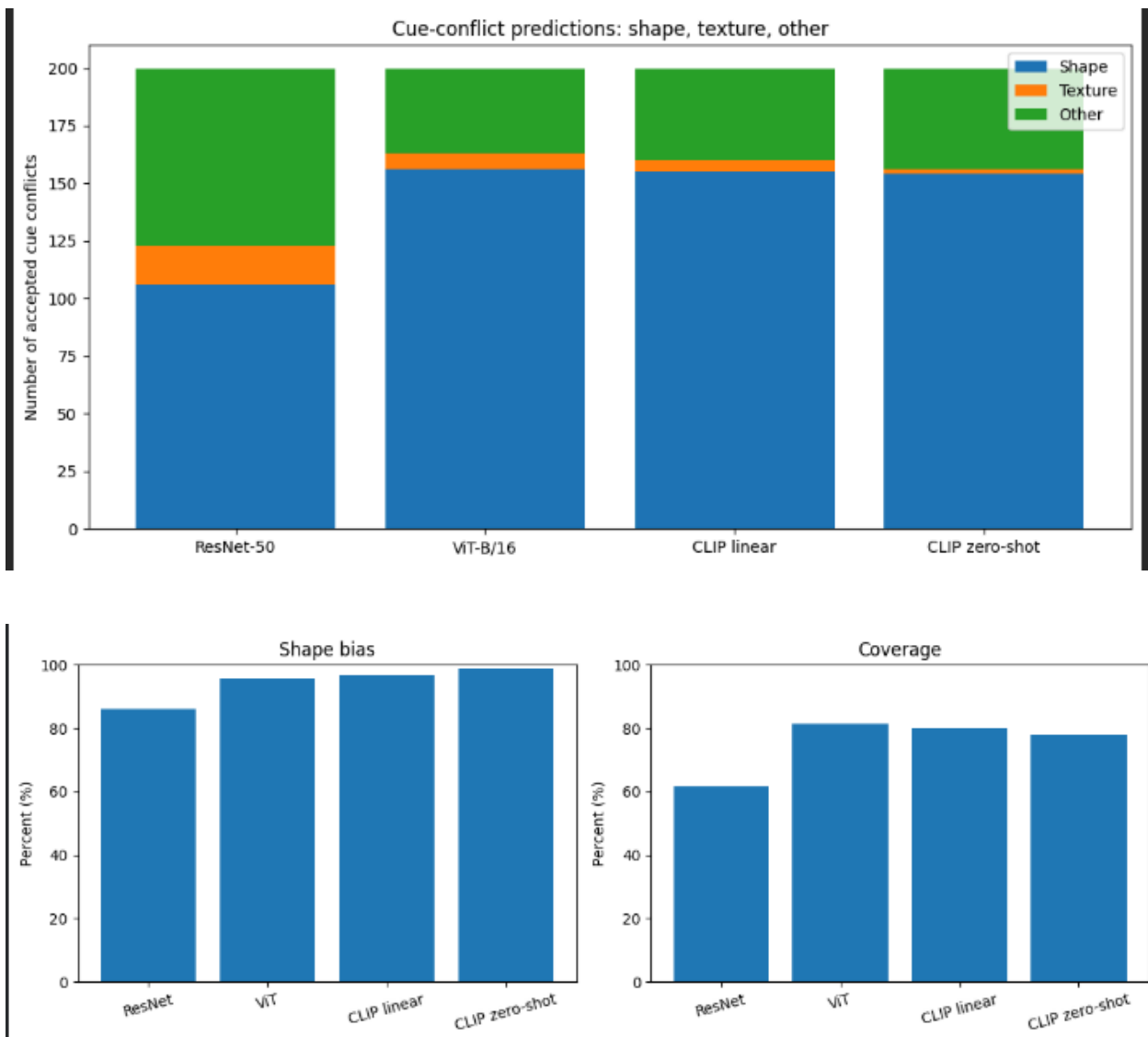


The results displayed that although consistency was reduced, the Hue rotation was more affected than the grey scale. This helps infer that the model is dependent but not fully on color, and this is portrayed by the dip in the bars of the graph. Hue rotation was more affected than grayscale, which portrays that even without color information, the model can predict correctly, but if given wrong information, it can decrease accuracy more.

E- Shape Bias

This experiment deliberately created image where it basically checks whether the model depends more on shape or texture. This was done by creating an image with the shape of an airplane and the texture of a cat. If the model predicts airplane, it's a shape decision, and if the model predicts cat, it's a texture-based decision. So, we create 5 pairs, including both directions, making it 10

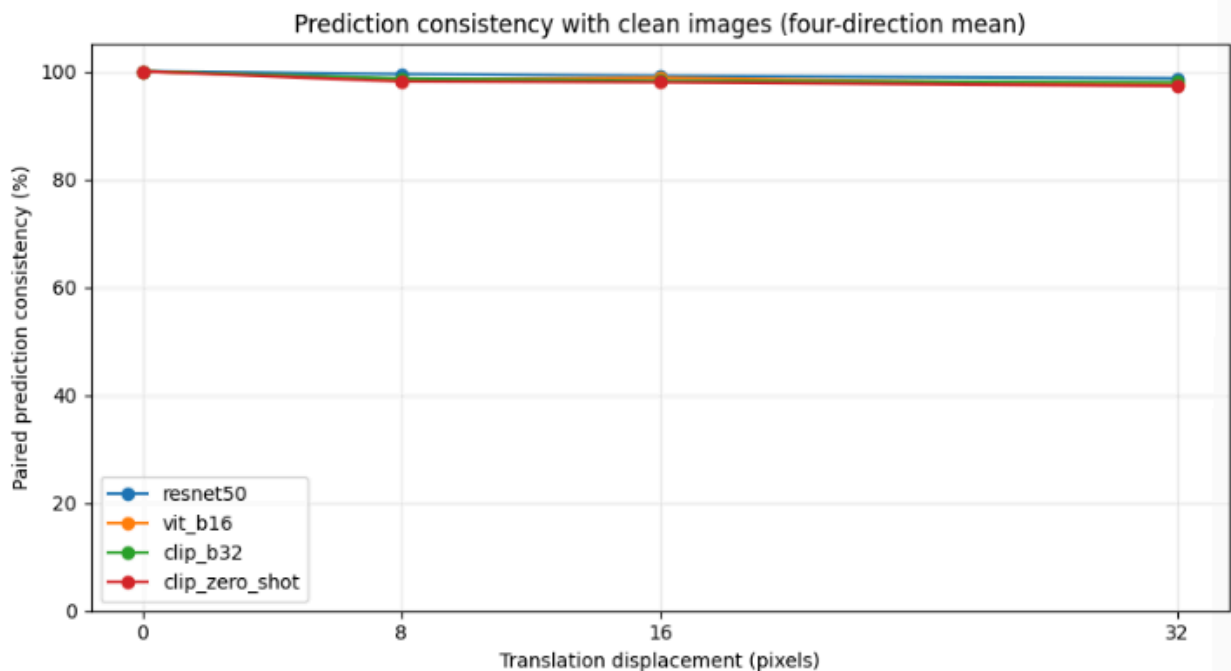
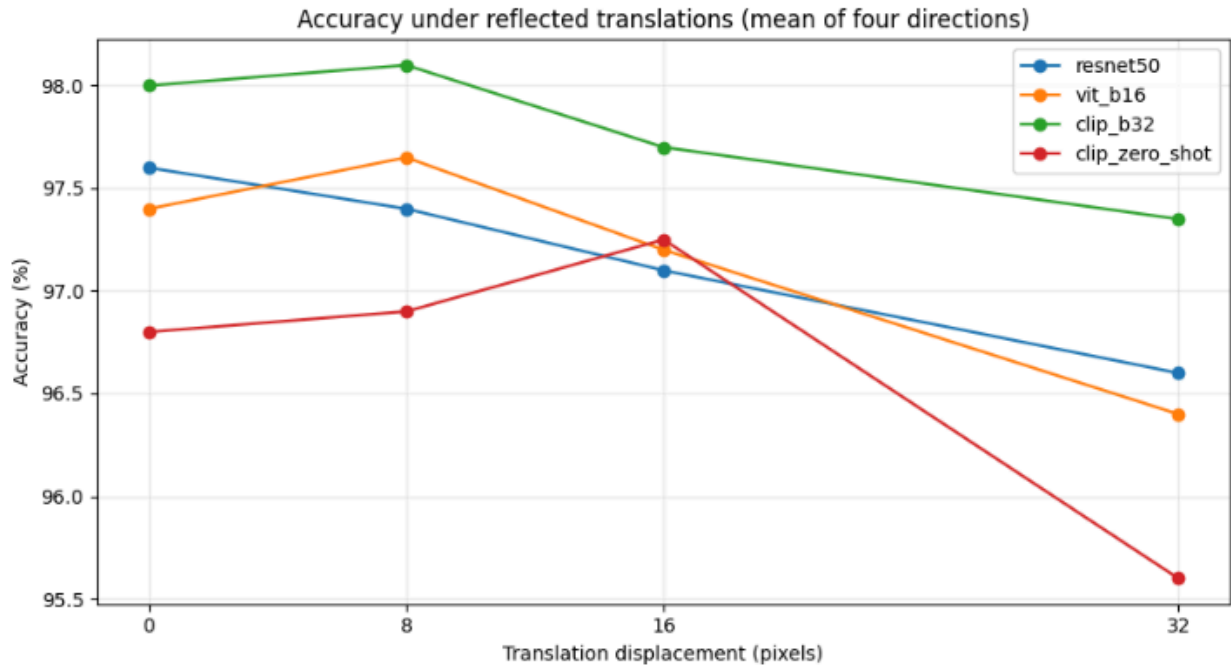
and 40 images each, creating 400 images, and then we use ADAIN normalization, which basically keeps the spatial arrangement of the content representation but makes each feature channel resemble the style representation. Visual screening classified 326 as valid images, rejected 52, and 22 were ambiguous. From that, a balanced subset of 200 accepted conflicts, 20 per pair-direction, was chosen for evaluation, and from those 200 images, we extract their representations and predict them using the modes as shape, texture, or other. Then find shape bias as shape/ shape + texture. It also calculates Coverage because what if the texture was 0 and the final answer is wrong.



ResNet strongly favored shape over texture when its prediction corresponded to either intended cue, but its relatively low 61.5% coverage indicates that a substantial fraction of cue-conflict images elicited unrelated class predictions. ViT and CLIP showed higher shape bias, along with higher coverage, indicating that predictions were more aligned towards shape content.

F - Translation:

This experiment basically checks whether the model remains consistent when the same object is displayed at another location, done using a translation operator. Adds padding using a reflected mirror



Task 2

A- setup

Sources = Photo, art, cartoon

Target = sketch

Allowed to see the target, but not its labels. Called UDA..

We use 24 sketches, so we use a batch size of each source, so $B_B = B_t$

The setup freezes batchnorm because if it's allowed to run, batchnorm will become another adaptive mechanism.

B - SourceOnly ERM

ERM - empirical risk minimization

$$L_{ERM} = -\frac{1}{B} \sum_{i=1}^B \log p(y_i | x_i).$$

$$\theta^* = \arg \min_{\theta} \frac{1}{3} (R_{Photo}(\theta) + R_{Art}(\theta) + R_{Cartoon}(\theta)).$$

Basically, L_{erm} is the loss of each image in the batch, such that the probability of the true label of that image given the image. In other words, it's the probability given by the model to the true label when the image is passed and then finding the ResNet parameters (θ) that minimizes the average classification loss across all 3 sources.

C -DAN with MMD alignment

DAN asks to make the source and target distributions statistically more similar.

DAN = Deep Adaptation Network

And it makes the source and target distributions closer by calculating loss as $L = \text{cross_entropy_loss} + \text{MMD alignment}$.

MMD here is basically

$$MMD^2 = E[k(x_s, x'_s)] + E[k(x_t, x'_t)] - 2E[k(x_s, x_t)].$$

Where K is

$$k(x, y) = \exp\left(-\frac{\|x - y\|^2}{b}\right)$$

B here is bandwidth or kernel standard deviation, controls how far apart the 2 feature vectors can be its similar

But the problem here is that it may make marginal distributions similar while aligning incorrect regions, and my results demonstrate this problem.

Domain separability decreased by making the source and target harder to distinguish, but the target accuracy decreased as well, which explains that better domain alignment doesn't portray better target recognition

D- DANN

Domain-Adversarial NN

Learned adversary: another NN trained to find the difference between 2 distributions

This uses a learned adversary rather than directly calculating MMD.

$$L_{domain} = -\frac{1}{N} \sum_i [d_i \log q_i + (1 - d_i) \log(1 - q_i)].$$

Basically, if the discriminator picks wrong, the loss increases, and if it picks correctly, it minimizes. Backbone's job is to fool the discriminator, and it does by maximizing the loss. It uses GRL (gradient reversal layer), which multiplies the gradient by a negative sign that maximizes the loss by making the backbone learn the features that make the source and target hard to distinguish.

Under the fixed assignment training protocol, DANN exhibited unstable adversarial optimization and failed to maintain class-discriminative source features.

The prescribed DANN exhibited severe numerical instability; gradient-norm clipping at 1.0 substantially improved source validation performance; reducing the LR further to $5e-5$ did not

improve stability/performance, so the clipped $1e-4$ configuration was retained based on source validation only.

E- CDAN

DAN only makes source and target domain closer but doesn't concern which feature a class belongs to. CDAN tries to make the alignment class-aware.

Basically at start, ResNet converts the image into learnable features and then classifier takes those features and produces 1 score for each class, and softmax converts it into probabilities. CDAN takes an outer product with the feature and probabilities and does the same discriminator stuff again for CDAN.

“ If domain discriminator is fooled, doesn't usually mean that backbone learnt useful domain-invariant features, it can mean that the representation collapsed”

Also, CDAN also relies on p (probabilities) early, if p is unreliable, it means that the adversarial alignment received poor semantic guidance.

F- Controlled DAN strength study

It isolates λ_{MMD} and changes to see how domain inseparability works. And it showed as the lambda increased, the domain separability decreases, meaning it became difficult to distinguish source and target. Thus, greater domain confusion produced worse target classification.

Also, weak MMD alignment increased overall sketch accuracy by 4.76%, but macro-F1 decreased, the increase in gain is not uniformly distributed across classes

Task 3

A -DAN-DG

There is no target. Align the 3 source domains with each other.

$$L_{DAN-DG} = L_{ERM} + \frac{\lambda_{DG}}{3} \sum_{e < e'} MMD^2(F(X_e), F(X_{e'})).$$

With three source domains, the sum is explicitly:

$$L_{DAN-DG} = L_{ERM} + \frac{\lambda_{DG}}{3} [MMD^2(P, A) + MMD^2(P, C) + MMD^2(A, C)].$$

Lambda_dg here controls how much importance to give domain alignment with respect to classification.

The point of domain generalization here is if the model removes Photo, Art and Cartoon specific information and keeps only information that is common across domains, like object identity features and therefore tries to transfer to sketch.

In simple words, keep causal features and remove spurious features

But, MMD doesn't know which features are the object identity features, it just tries to make the distribution similar. Therefore excessive alignment can make objects like dog, horse less separable.

It also asks how easy it is to figure out which image came from which source domain?

A high score will mean that domain identity is still intact, i.e. 47.62 % domain separability.

But classification reduced from 94% to 22% meaning it destroyed a huge amount of useful class information.

B - SAM : Sharpness Aware Minimization

Basically find parameters that can keep loss consistent even if I slightly perturb them

It wants to basically avoid sharp Losses which has a huge steep curve if we were to move theta and it does this by

Compute:

$$g = \nabla_{\theta} L_{val}(\theta).$$

Then:

$$\epsilon = 0.05 \frac{g}{\|g\|_2}.$$

And measure:

$$\Delta_{\text{sharp}} = L_{val}(\theta + \epsilon) - L_{val}(\theta).$$

If the loss jumps a lot, that means the loss curve was sharp at that point

SAM reduces the local sharpness measure substantially while preserving classification. Though SAM doesn't separate domains properly, it generalizes very well.

Task 4 -Open Set Recognition

Basically, this is that if we were to train our model on known classes and if we were to deploy the model into an open set, there might exist unknown classes, and this tells the methods to cater to them

A - Setup

Basically, we have known CIFAR-10 data, and unknown CIFAR-10 classes, and each side has 800 images each.

Near_Unknown: wolf

When CIFAR-10 contains dog, cat.. semantically very similar (VERY DIFFICULT TO REJECT)

Far_Unknown: keyboard Not closely similar semantically

So, these are far easier to reject

B- Vanilla Closed Set Baseline

This basically explains why the ResNet-18 architecture is slightly modified when training on CIFAR-10 dataset, because a normal ResNet is trained for ImageNet whose dimension are basically 224x224

But CIFAR-10 images are 32x32

So, if we were to use ImageNet ResNet-18 input layers, we had to shrink the CIFAR image too aggressively before the model even has the chance to extract information.

“Stride = 1 means moving 1 pixel at a time”

Thus, we reduce the kernel of ResNet from 7x7, stride = 2 to 3x3, stride = 1 and no maxpool, Which preserves the spatial resolution at the beginning

Dataset is split as 45k training and 5k for validation, thus stratified 4.5k/train + 0.5k/val per class

Vanilla Training: Normal Cross-Entropy loss, which is given as 95%.

– Post-hoc Novelty scores:

In other words, after training the model, give an unknown score to each unknown image

The ways to measure unknown score:

1. MSP [Maximum Softmax Probability]:
1 - max probability score of the logit
The lower the score, the unknown image is
2. MLS [Maximum logit Score]
-max logit score
The lower the MLS is, the known the image is

MLS is better than MSP because it retains the absolute evidence magnitude that softmax normalization partially removes

3. Energy:
Uses all logit

$$u_{Energy}(x) = -\log \sum_{k=1}^{10} e^{z_k(x)}$$

Lower the energy, the more known the image is

4. Mahalanobis distance:
Basically calculate the mean of a feature vector and then find the distance of the unknown from all the clusters.. The minimum distance to the known cluster is called mahalanobis distance.
If the value is small, meaning the less unknown the image is

$$u_{Mah}(x) = \min_c (f(x) - \mu_c)^T \Sigma^{-1} (f(x) - \mu_c)$$

Σ^{-1} is the covariance matrix.

To distinguish whether the image is known or unknown, we use threshold. If the score is greater than threshold, we can say that it can be known or unknown

AUROC: Across every possible threshold, how well does the score rank unknown above knowns?

C- GCSC

Good closed set classifier

Maybe open-set recognition becomes better simply if the underlying closed set classifier becomes better.

Vanilla: RandomCrop + horizontal flip

GCSC: RandomCrop + horizontal flip + RandArgument(2,9)

Randargument(2,9) applies stronger image transformations during training.

2 -> 2 augmentation operations

9 -> Strength of transformation

Basically forces the classifier to recognize CIFAR-10 objects under more visual variation

Improvement in accuracy by only 0.21%

Rejects Unknowns more profoundly

So, stronger augmentation barely increase accuracy but creates a better unknown separation

D- PROSER

Placeholder for OpenSet-Recognition

Creates unknown regions using 2 kinds of placeholders. Basically we have 10 classes and we create 5 dummy classes and this is done by taking a pair of 2 classes and finding a middle region between them such that region doesnot belong to either class and classify it as dummy

2 objectives:

1. If for a known example, it should classify as the correct class, meaning the logit of dog should be highest if the correct example's true label is dog
2. If we were to remove class label "dog" and test the same example, this means that now the known example should classify as dummy.

In my notebook, PROSER didnt improve unknown rejection. This means that under this CIFAR-10 split and required training, the learned placeholders were not able to outperform the simpler baselines.

PROSER might fail because the real unknown classes may not necessarily lie on the dummies
In other words, synthetic unknowns are not equal to all possible real unknowns.